

Berikut adalah penjelasan mendalam mengenai kode `OperationManager.cs`. Kode ini bertindak sebagai "**Jembatan**" (**Controller**) yang menghubungkan antara Tampilan (UI Form), Data Gambar (Service), dan Logika Matematika Gambar (Arithmetic/Logical Operations).

Secara garis besar, kelas ini bertugas:

1. Mengambil gambar dari kanvas utama (Gambar A).
2. Mencari gambar kedua (Gambar B) berdasarkan `RadioButton` yang dipilih user.
3. Memastikan kedua gambar valid.
4. Mengirim gambar tersebut ke fungsi pemroses (seperti `Add`, `Subtract`, dll).
5. Mengembalikan hasilnya ke Form utama.

1. Library & Namespace

C#

```
using MiniPhotoshop.Logic;  
using MiniPhotoshop.Logic.ImageProcessing;  
using System.Collections.Generic;  
using System.Drawing;  
using System.Globalization;  
using System.Windows.Forms;
```

```
namespace MiniPhotoshop.Helpers
```

- **MiniPhotoshop.Logic...**: Mengimpor logika pemrosesan gambar yang sudah Anda buat di file lain (misal kelas `ArithmeticOperations`).
- **System.Drawing**: Wajib ada karena kita memanipulasi objek `Bitmap` (gambar).
- **System.Globalization**: Digunakan nanti untuk membaca angka desimal (koma atau titik) pada `TextBox` dengan benar.
- **System.Windows.Forms**: Diperlukan karena kelas ini berinteraksi langsung dengan kontrol UI seperti `RadioButton`, `PictureBox`, dan `TextBox`.

2. Deklarasi Kelas & Variabel Global (Fields)

C#

```
public class OperationManager
{
    private readonly ImageEditorService _editorService;
    private readonly Dictionary<RadioButton, PictureBox> _thumbnailMap;
    private readonly TextBox _constantTextBox;
```

- **_editorService:** Ini adalah referensi ke layanan utama yang menyimpan gambar yang sedang diedit di kanvas utama.
- **_thumbnailMap:** Ini adalah perubahan logika utama Anda. Alih-alih mengecek satu per satu secara manual, kita menggunakan Dictionary (Kamus).
 - **Kunci (Key):** RadioButton (Tombol pilihan).
 - **Nilai (Value):** PictureBox (Gambar thumbnail terkait).
 - *Konsep:* "Jika RadioButton X dipilih, maka ambil gambar dari PictureBox Y".
- **_constantTextBox:** Referensi ke kotak teks tempat user mengetik nilai konstanta (misalnya "1.5" untuk perkalian).

3. Constructor (Metode Pembangun)

C#

```
public OperationManager(ImageEditorService editorService,
    Dictionary<RadioButton, PictureBox> thumbnailMap,
    TextBox constantTextBox)
{
    _editorService = editorService;
    _thumbnailMap = thumbnailMap;
    _constantTextBox = constantTextBox;
}
```

- Saat OperationManager dibuat (di MainForm), ia meminta 3 hal tersebut. Ini disebut **Dependency Injection**. Kelas ini tidak membuat komponen UI sendiri, tapi "diberi" komponen yang sudah ada dari Form utama untuk dikelola.

4. Logika Inti Pemilihan Gambar B (GetImageB_FromRadioButtons)

Ini adalah fungsi helper baru yang sangat penting.

C#

```
private Bitmap GetImageB_FromRadioButtons()
{
    // Loop (berputar) memeriksa setiap pasangan RadioButton dan PictureBox di dalam kamus
    foreach (var pair in _thumbnailMap)
    {
        RadioButton rb = pair.Key;    // Ambil tombolnya
        PictureBox pb = pair.Value;    // Ambil picturebox pasangannya

        // Logika: Apakah tombol ini dicentang? DAN Apakah PictureBox-nya punya gambar?
        if (rb.Checked && pb.Image != null)
        {
            return new Bitmap(pb.Image); // Ketemu! Salin gambarnya dan kembalikan.
        }
    }

    // Jika loop selesai tapi tidak ada yang cocok (tidak ada yang dipilih/gambar kosong)
    return null;
}
```

- **Fungsinya:** Mencari tahu gambar mana yang harus dijadikan "Gambar B" (pengurang/pembagi/dll) tanpa perlu menulis if-else yang panjang untuk setiap tombol.

5. Validasi Operand (GetImageOperands)

Fungsi ini memastikan operasi matematika aman dilakukan (mencegah error/crash).

C#

```
private bool GetImageOperands(out Bitmap imgA, out Bitmap imgB)
{
    imgA = null;
    imgB = null;

    // 1. Cek Gambar A (Kanvas Utama)
```

```

if (!_editorService.IsImageLoaded)
{
    // Tampilkan pesan error jika kanvas kosong
    MessageBox.Show(..., "Gambar A Kosong", ...);
    return false; // Operasi gagal
}

// 2. Cek Gambar B (Thumbnail)
imgB = GetImageB_FromRadioButtons(); // Panggil fungsi helper di poin 4
if (imgB == null)
{
    // Tampilkan pesan error jika user lupa memilih RadioButton atau memilih yang kosong
    MessageBox.Show(..., "Gambar B Kosong", ...);
    return false; // Operasi gagal
}

// 3. Jika semua aman, ambil Gambar A
imgA = _editorService.GetRestoredImage();
return true; // Operasi berhasil disiapkan
}

```

- **out Bitmap imgA, out Bitmap imgB:** Kata kunci out berarti fungsi ini tidak hanya mengembalikan true/false, tapi juga "mengeluarkan" dua objek gambar (imgA dan imgB) untuk dipakai oleh pemanggilnya.

6. Operasi Aritmatika & Logika (Contoh: PerformAdd)

Semua fungsi Perform... (Add, Subtract, And, Or) memiliki pola yang sama persis.

C#

```

public Bitmap PerformAdd()
{
    // 1. Cek apakah A dan B ada? (Memanggil validasi di poin 5)
    if (GetImageOperands(out Bitmap imgA, out Bitmap imgB))
    {
        // 2. Blok 'using' untuk Manajemen Memori
        using (imgA) using (imgB)
        {

```

```

        // 3. Panggil Logika Matematika
        return ArithmeticOperations.Add(imgA, imgB);
    }
}
// Jika validasi gagal, kembalikan gambar asli (tidak ada perubahan)
return GetOriginalImage();
}

```

- **Pentingnya using:** Gambar (Bitmap) di C# memakan memori sistem yang besar (GDI+). Perintah using memastikan bahwa setelah operasi penjumlahan selesai, memori untuk imgA dan imgB (yang merupakan salinan sementara) langsung dibersihkan/dibuang agar aplikasi tidak berat (memory leak).
- **ArithmeticOperations.Add:** Ini memanggil kode statis yang ada di file logika lain (yang melakukan loop pixel r,g,b).

7. Operasi NOT (PerformNot)

Operasi NOT (Invert) sedikit berbeda karena hanya butuh 1 gambar (Gambar A).

C#

```

public Bitmap PerformNot()
{
    // Cukup cek Gambar A saja
    if (!_editorService.IsImageLoaded) { ... error ... }

    using (Bitmap imgA = GetOriginalImage())
    {
        return LogicalOperations.Not(imgA);
    }
}

```

8. Helper Konstanta (TryGetConstant)

Ini menangani input angka dari TextBox user.

C#

```
private bool TryGetConstant(out double valueK)
{
    // Mencoba mengubah Teks menjadi Angka (Double)
    // NumberStyles.Any & CultureInfo.InvariantCulture:
    // Memastikan "1.5" terbaca sebagai satu koma lima, terlepas dari setting komputer (ID/US).
    if (double.TryParse(_constantTextBox.Text, NumberStyles.Any,
        CultureInfo.InvariantCulture, out valueK))
    {
        return true; // Berhasil konversi
    }

    // Jika user mengetik huruf "abc" atau format salah
    MessageBox.Show("Nilai Konstanta (K) tidak valid...", "Input Error", ...);
    valueK = 1.0; // Nilai default biar tidak crash
    return false;
}
```

9. Operasi Konstanta (PerformConstantMultiply)

Melakukan operasi perkalian/pembagian dengan angka skalar (bukan dengan gambar lain).

C#

```
public Bitmap PerformConstantMultiply()
{
    // 1. Ambil nilai K dari TextBox (validasi dulu)
    if (!TryGetConstant(out double k)) return GetOriginalImage();

    // 2. Ambil Gambar A
    using (Bitmap imgA = GetImageA())
    {
        if (imgA == null) return null;
        // 3. Lakukan operasi (Gambar * Angka)
        return ConstantOperations.Multiply(imgA, k);
    }
}
```

```
}
```

Kesimpulan Alur Kerja

1. User klik tombol "Add" di Form.
2. Form memanggil `operationManager.PerformAdd()`.
3. `OperationManager` mengecek `_thumbnailMap` untuk melihat `RadioButton` mana yang aktif.
4. Jika valid, `OperationManager` mengambil kedua gambar.
5. Gambar dikirim ke `ArithmeticOperations.Add`.
6. Hasil gambar (`Bitmap` baru) dikembalikan ke Form untuk ditampilkan.